

DAY 6

Akash J | 192324295

Dynamic Programming – Graph Algorithms (Bellman-Ford, Floyd-Warshall, Warshall's)

1. Bellman-Ford Algorithm

Aim:

To find the shortest distance from the source to all other vertices and detect negative weight cycles.

Algorithm:

1. Initialize distances from source to all vertices as infinity, and source as 0.
2. Relax all edges $V-1$ times.
3. Check for negative weight cycles by relaxing one more time.

Code:

```
def bellman_ford(V, edges, src):
    dist = [float('inf')] * V
    dist[src] = 0
    for _ in range(V - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            print('Negative Weight Cycle Detected')
            return
    print('Vertex    Distance')
    for i in range(V):
        print(f'{i}          {dist[i]}')
```

Input:

```
Vertices = 5
Edges = [(0,1, -1), (0,2,4), (1,2,3), (1,3,2), (1,4,2), (3,2,5), (3,1,1), (4,3, -3)]
Source = 0
```

Output:

```
Vertex Distance
0 0
1 -1
2 2
3 -2
4 1
```

Result: Shortest distances calculated successfully.

2. Floyd-Warshall Algorithm

Aim:

To find the shortest distances between every pair of vertices.

Algorithm:

1. Initialize the solution matrix same as input graph matrix.
2. Update the solution matrix by considering all vertices as an intermediate vertex.
3. Output the updated matrix.

Code:

```
def floyd_warshall(graph):
    V = len(graph)
    dist = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                if dist[i][k] != float('inf') and dist[k][j] != float('inf'):
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    for row in dist:
        print(['INF' if x == float('inf') else x for x in row])
```

Input:

```
Graph:
[0, 5, INF, 10]
[INF, 0, 3, INF]
[INF, INF, 0, 1]
[INF, INF, INF, 0]
```

Output:

```
[0, 5, 8, 9]
['INF', 0, 3, 4]
['INF', 'INF', 0, 1]
['INF', 'INF', 'INF', 0]
```

Result: All-pairs shortest paths successfully computed.

3. Warshall's Algorithm

Aim:

To determine the transitive closure of a directed graph.

Algorithm:

1. Initialize reachability matrix same as adjacency matrix.
2. For every vertex k, check if path exists from i to j through k.
3. Print the final reachability matrix.

Code:

```
def warshall(graph):  
    V = len(graph)  
    reach = [row[:] for row in graph]  
    for k in range(V):  
        for i in range(V):  
            for j in range(V):  
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])  
    for row in reach:  
        print(row)
```

Input:

```
Graph:  
[0,1,0,0]  
[0,0,1,0]  
[0,0,0,1]  
[0,0,0,0]
```

Output:

```
[0, 1, 1, 1]  
[0, 0, 1, 1]  
[0, 0, 0, 1]  
[0, 0, 0, 0]
```

Result: Transitive closure successfully computed.

4. Find the City with the Smallest Number of Reachable Cities

Aim:

To find the city that can reach the smallest number of other cities within a threshold distance.

Algorithm:

1. Use Floyd-Warshall to find shortest paths between all cities.
2. Count reachable cities for each city within threshold.
3. Return the city with the minimum count (tie-breaking with largest index).

Code:

```
def findTheCity(n, edges, distanceThreshold):
    dist = [[float('inf')] * n for _ in range(n)]
    for i in range(n): dist[i][i] = 0
    for u, v, w in edges:
        dist[u][v] = dist[v][u] = w
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    ans, min_reach = -1, n
    for i in range(n):
        reach = sum(dist[i][j] <= distanceThreshold for j in range(n))
        if reach <= min_reach:
            min_reach = reach
            ans = i
    print(ans)
```

Input:

```
n=4
edges=[[0,1,3],[1,2,1],[2,3,4],[0,3,7]]
threshold=4
```

Output:

3

Result: City computed successfully.

5. Network Delay Time

Aim:

To find the minimum time for all nodes to receive a signal using Bellman-Ford.

Algorithm:

1. Initialize distances with infinity, source distance with 0.
2. Relax all edges V-1 times.
3. Return the maximum distance. If infinity exists, return -1.

Code:

```
def networkDelayTime(times, n, k):
    dist = [float('inf')] * (n + 1)
    dist[k] = 0
    for _ in range(n - 1):
        for u, v, w in times:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    max_dist = max(dist[1:])
    print(max_dist if max_dist != float('inf') else -1)
```

Input:

```
times=[[2,1,1],[2,3,1],[3,4,1]]
n=4
k=2
```

Output:

```
2
```

Result: Network delay computed successfully.

6. Cheapest Flight Within K Stops

Aim:

To find the minimum cost to travel from source to destination with at most K stops.

Algorithm:

1. Initialize distances with infinity.
2. Relax edges up to K+1 times using a temporary distance array to limit stops.
3. Return the destination distance.

Code:

```
def findCheapestPrice(n, flights, src, dst, K):
    dist = [float('inf')] * n
    dist[src] = 0
    for i in range(K + 1):
        temp = dist[:]
        for u, v, w in flights:
            if dist[u] != float('inf') and dist[u] + w < temp[v]:
                temp[v] = dist[u] + w
        dist = temp
    ans = dist[dst] if dist[dst] != float('inf') else -1
    print(ans)
```

Input:

```
n = 4
flights = [[0,1,100],[1,2,100],[2,3,100],[0,3,500]]
src = 0, dst = 3, K = 1
```

Output:

```
500
```

Result: Cheapest flight computed successfully.

7. Detect Negative Weight Cycle

Aim:

To determine whether a graph contains a negative weight cycle.

Algorithm:

1. Initialize distances from 0.
2. Relax V-1 times.
3. Relax one more time; if a distance changes, a negative cycle exists.

Code:

```
def detect_cycle(V, edges):
    dist = [0] * V
    for _ in range(V - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            print('Negative Weight Cycle Exists')
            return
    print('No Negative Weight Cycle')
```

Input:

```
Vertices = 4
Edges = [(0,1,1),(1,2,-1),(2,3,-1),(3,0,-1)]
```

Output:

```
Negative Weight Cycle Exists
```

Result: Negative cycle detection completed successfully.

8. Determine Reachability Between Two Vertices

Aim:

To determine whether a path exists between u and v using Warshall's Algorithm.

Algorithm:

1. Compute the transitive closure of the graph.
2. Check if the value at $\text{matrix}[u][v]$ is 1.
3. Print 'Path Exists' or 'Path Does Not Exist'.

Code:

```
def check_path(graph, u, v):
    V = len(graph)
    reach = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    if reach[u][v]:
        print('Path Exists')
    else:
        print('Path Does Not Exist')
```

Input:

```
Graph = [[0,1,0], [0,0,1], [0,0,0]]
u = 0, v = 2
```

Output:

```
Path Exists
```

Result: Reachability correctly determined.

9. Count Reachable Vertices from a Source

Aim:

To find the number of vertices reachable from a specified source.

Algorithm:

1. Generate transitive closure.
2. Count the number of 1s in the source's row.
3. Exclude the source vertex itself from the count.

Code:

```
def count_reachable(graph, src):
    V = len(graph)
    reach = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    count = sum(reach[src]) - reach[src][src]
    print(count)
```

Input:

```
Graph = [[0,1,0,0], [0,0,1,0], [0,0,0,1], [0,0,0,0]]
Source = 0
```

Output:

3

Result: Reachable vertices counted successfully.

10. Find the Diameter of a Graph

Aim:

To find the diameter (maximum shortest-path distance) of a graph.

Algorithm:

1. Run Floyd-Warshall to get shortest paths.
2. Find the maximum value in the resulting matrix excluding infinity.
3. Output the diameter.

Code:

```
def graph_diameter(graph):
    V = len(graph)
    dist = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    diameter = 0
    for i in range(V):
        for j in range(V):
            if dist[i][j] != float('inf') and i != j:
                diameter = max(diameter, dist[i][j])
    print(diameter)
```


Input:

```
Graph = [[0,2,INF], [2,0,3], [INF,3,0]]
```

Output:

5

Result: Diameter found successfully.

11. Find the Shortest Distance Between Two Vertices

Aim:

To find the shortest distance between a specific pair of vertices u and v.

Algorithm:

1. Compute all-pairs shortest paths using Floyd-Warshall.
2. Extract the value at `dist[u][v]`.
3. Output the result.

Code:

```
def shortest_dist(graph, u, v):  
    V = len(graph)  
    dist = [row[:] for row in graph]  
    for k in range(V):  
        for i in range(V):  
            for j in range(V):  
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])  
    print(dist[u][v])
```

Input:

```
Graph = [[0,3,INF,7], [8,0,2,INF], [5,INF,0,1], [2,INF,INF,0]]  
u = 0, v = 3
```

Output:

6

Result: Shortest distance between pair correctly found.

12. Find a Vertex Reachable from Every Other Vertex

Aim:

To find the smallest indexed vertex reachable from all other vertices.

Algorithm:

1. Compute transitive closure using Warshall's.
2. Iterate through columns of the result.
3. If a column has all 1s (or all other vertices can reach it), return its index.

Code:

```
def reachable_from_all(graph):
    V = len(graph)
    reach = [row[:] for row in graph]
    for i in range(V): reach[i][i] = 1
    for k in range(V):
        for i in range(V):
            for j in range(V):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    for j in range(V):
        if all(reach[i][j] for i in range(V)):
            print(j)
            return
    print(-1)
```

Input:

```
Graph = [[0,1,1], [0,0,1], [0,0,0]]
```

Output:

```
2
```

Result: Common reachable vertex successfully found.

13. Find the Vertex with Maximum Reachability

Aim:

To find the vertex that can reach the highest number of other vertices.

Algorithm:

1. Compute the transitive closure.
2. Sum the rows of the reachability matrix.
3. Return the index with the maximum sum.

Code:

```
def max_reachability(graph):
    V = len(graph)
    reach = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    max_reach = -1
    ans = -1
    for i in range(V):
        cur_reach = sum(reach[i])
        if cur_reach > max_reach:
            max_reach = cur_reach
            ans = i
    print(ans)
```

Input:

```
Graph = [[0,1,1,0], [0,0,1,0], [0,0,0,1], [0,0,0,0]]
```

Output:

```
0
```

Result: Vertex with max reachability computed.

14. Find the Diameter of a Weighted Graph

Aim:

To find the maximum shortest-path distance in a weighted graph.

Algorithm:

1. Apply Floyd-Warshall Algorithm.
2. Find the maximum shortest distance across all valid pairs.
3. Output the result.

Code:

```
def weighted_diameter(graph):
    V = len(graph)
    dist = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    diameter = 0
    for i in range(V):
        for j in range(V):
            if dist[i][j] != float('inf') and i != j:
                diameter = max(diameter, dist[i][j])
    print(diameter)
```

Input:

```
Graph = [[0,2,INF], [2,0,4], [INF,4,0]]
```

Output:

```
6
```

Result: Weighted diameter computed successfully.

15. Check Whether a Graph is Strongly Connected

Aim:

To determine if every vertex can reach every other vertex.

Algorithm:

1. Run Warshall's Algorithm.
2. Check if the resulting reachability matrix contains all 1s.
3. Output 'Strongly Connected' or 'Not Strongly Connected'.

Code:

```
def strongly_connected(graph):
    V = len(graph)
    reach = [row[:] for row in graph]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    for i in range(V):
        for j in range(V):
            if i != j and not reach[i][j]:
                print('Not Strongly Connected')
                return
    print('Strongly Connected')
```

Input:

```
Graph = [[0,1,0], [0,0,1], [1,0,0]]
```

Output:

```
Strongly Connected
```

Result: Strong connectivity verified successfully.